

9강

spring Boot  
JDBC 연동하기

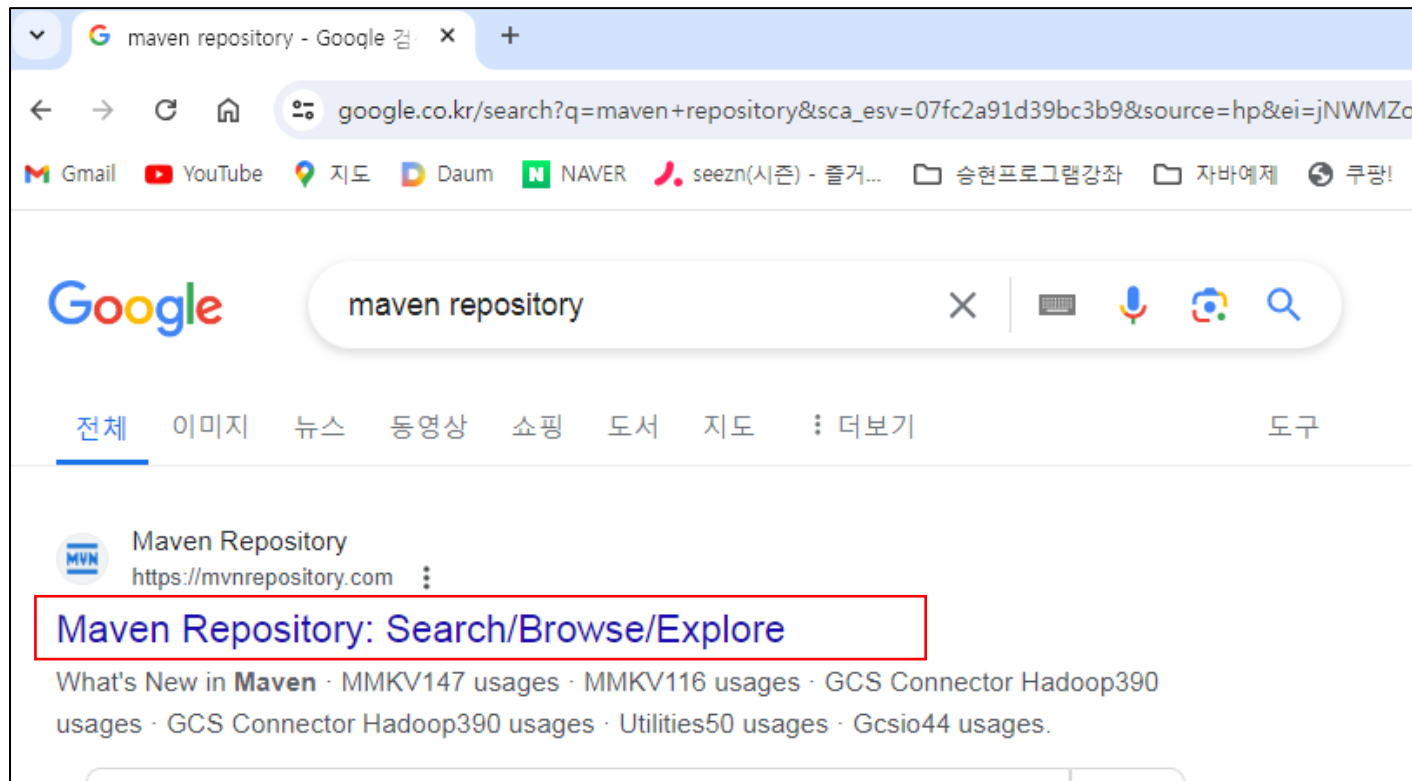




spring

## ➤ MySQL JDBC로 연결 하기

<https://mvnrepository.com/>





## spring

- MySQL 스프링 JDBC로 연결 하기  
Spring boot starter jdbc => 검색창에 입력한다.

The screenshot shows a web browser window with the URL `mvnrepository.com/search?q=Spring+boot+starter+jdbc`. The page displays the Maven Repository search results for the query. The search bar contains the text "Spring boot starter jdbc". The results are sorted by Relevance. The first result, "1. Spring Boot Starter JDBC", is highlighted with a red box. It shows the package `org.springframework.boot » spring-boot-starter-jdbc`, a description "Starter for using JDBC with the HikariCP connection pool", and the last release date "Jan 22, 2026". The second result, "2. Spring Boot Starter Data JDBC", is also visible below it. On the left side, there is a "Filters" section with a "Repository" filter showing various repositories like Central, Sonatype, Mulesoft, etc.

Maven Repository: Spring boot starter jdbc

mvnrepository.com/search?q=Spring+boot+starter+jdbc

Spring boot starter jdbc

Found 53535 results

Sort by: Relevance

1. **Spring Boot Starter JDBC**  
org.springframework.boot » spring-boot-starter-jdbc  
Starter for using JDBC with the HikariCP connection pool  
Last Release on Jan 22, 2026  
1,977 usages  
Apache

2. **Spring Boot Starter Data JDBC**  
org.springframework.boot » spring-boot-starter-data-jdbc  
Starter for using Spring Data JDBC  
Last Release on Jan 22, 2026  
205 usages  
Apache

Filters

Repository

- Central 45.8k
- Sonatype 2.4k
- Mulesoft 1.8k
- JCenter 1.5k
- Spring Plugins 1.5k
- Spring Lib M 1.3k
- Alfresco 841
- Cloudera Pub 775

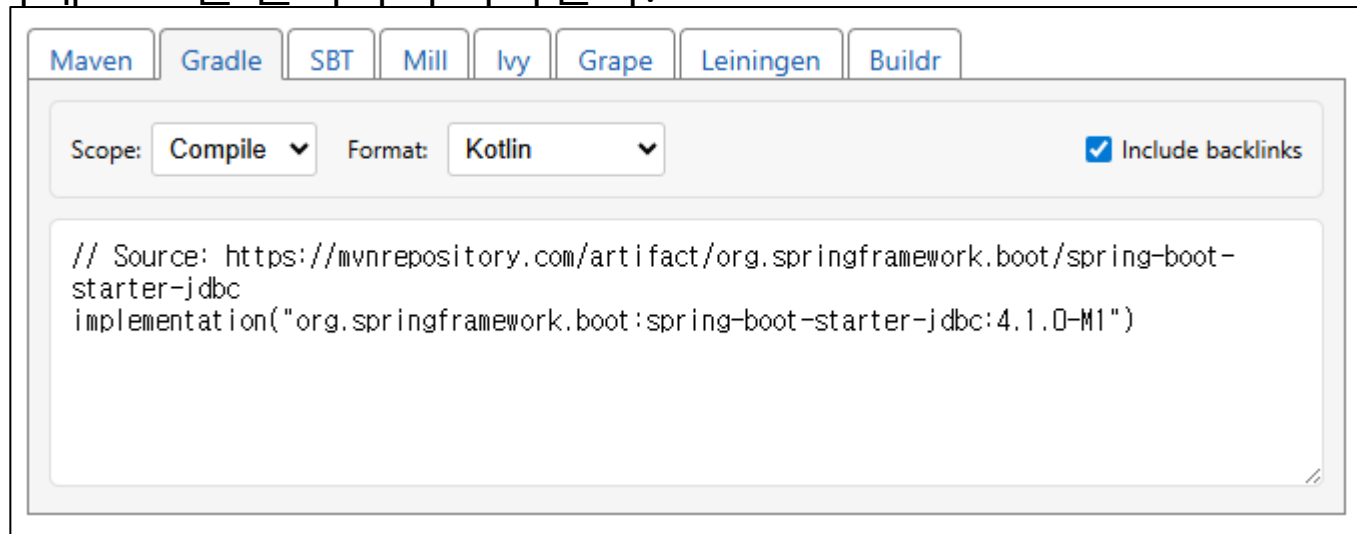
Group



## spring

## ➤ MySQL 스프링 JDBC로 연결 하기

아래 코드를 클릭하여 복사한다.

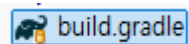


```
// Source:  
https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-jdbc  
implementation("org.springframework.boot:spring-boot-starter-jdbc:4.1.0-M1")
```



## spring

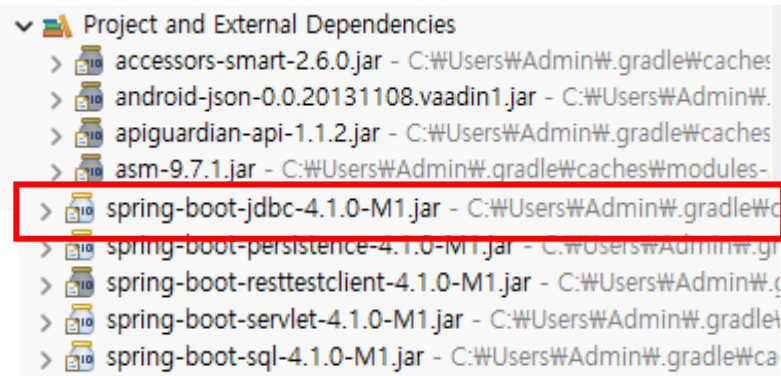
## ➤ MySQL 스프링 JDBC로 연결 하기



Build.gradle 더블클릭하여 의존성 주입한다.

```
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'  
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'  
  
    // Source: https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-jdbc  
    implementation("org.springframework.boot:spring-boot-starter-jdbc")  
  
    developmentOnly 'org.springframework.boot:spring-boot-devtools'  
    testImplementation 'org.springframework.boot:spring-boot-starter-thymeleaf-test'  
    testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'  
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'  
}
```

반드시 프로젝트 위에서 Gradle -> Refresh Gradle Project 를 진행한다.





spring

- MySQL 스프링 JDBC로 연결 하기  
동일한 방법으로 'com.mysql.mysql-connector-j' 의존성을 프로젝트에 추가

The screenshot shows the Maven Repository search results for the query 'com.mysql.mysql-connector-j'. The search bar at the top contains the query, and the results show 9763 results. The first result, 'MySQL Connector/J', is highlighted with a red box. It is a JDBC Type 4 driver, which means it is a pure Java implementation of the MySQL protocol and does not rely on the MySQL client libraries. This driver supports auto-registration with the Driver Manager, standardized validity ... The last release was on Oct 27, 2025. The second result, 'MySQL Connector Java', is also visible below it.

**MVN REPOSITORY** com.mysql.mysql-connector-j Search

Found 9763 results Sort by: Relevance ▼

**Filters**

Repository

- Central 6.8k
- Mulesoft 799
- Mulesoft Releases 474
- Sonatype 411
- BeDataDriven 314
- WSO2 Public 234
- JCenter 230
- JBoss Repo 219

Group

**1. MySQL Connector/J** 1,484 usages

com.mysql » mysql-connector-j

MySQL Connector/J is a JDBC Type 4 driver, which means that it is pure Java implementation of the MySQL protocol and does not rely on the MySQL client libraries. This driver supports auto-registration with the Driver Manager, standardized validity ...

Last Release on Oct 27, 2025

**2. MySQL Connector Java** 8,639 usages

mysql » mysql-connector-java

MySQL Connector/J is a JDBC Type 4 driver, which means that it is pure Java implementation of the MySQL protocol and does not rely on the MySQL client libraries. This driver supports auto-registration with the Driver Manager, standardized validity ...



## ➤ MySQL 스프링 JDBC로 연결 하기

현재 설치된 MySQL 버전을 확인하고 클릭한다. ( version 8.0.32)

[Home](#) » [com.mysql](#) » [mysql-connector-j](#)



### MySQL Connector/J

MySQL Connector/J is a JDBC Type 4 driver, which means that it is pure Java implementation of the MySQL protocol and does not rely on the MySQL client lib with the Driver Manager, standardized validity checks, categorized SQLExceptions, support for large update counts, support for local and offset date-time vari JDBC-4.x XML processing, support for per connection client information and support for the NCHAR, NVARCHAR ...

|            |                                                                                                                                                                                           |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Categories | JDBC Drivers                                                                                                                                                                              |
| Tags       | <a href="#">database</a> <a href="#">sql</a> <a href="#">jdbc</a> <a href="#">driver</a> <a href="#">connector</a> <a href="#">rdbms</a> <a href="#">mysql</a> <a href="#">connection</a> |
| Ranking    | #712 in MvnRepository (See Top Artifacts)<br>#9 in JDBC Drivers                                                                                                                           |
| Used By    | 705 artifacts                                                                                                                                                                             |

Central (8)

Redhat GA (1)

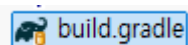
Redhat EA (1)

| Version |                        | Vulnerabilities | Repository              |
|---------|------------------------|-----------------|-------------------------|
| 9.0.x   | <a href="#">9.0.0</a>  |                 | <a href="#">Central</a> |
| 8.4.x   | <a href="#">8.4.0</a>  |                 | <a href="#">Central</a> |
| 8.3.x   | <a href="#">8.3.0</a>  |                 | <a href="#">Central</a> |
| 8.2.x   | <a href="#">8.2.0</a>  |                 | <a href="#">Central</a> |
| 8.1.x   | <a href="#">8.1.0</a>  |                 | <a href="#">Central</a> |
|         | <a href="#">8.0.33</a> |                 | <a href="#">Central</a> |
| 8.0.x   | <a href="#">8.0.32</a> |                 | <a href="#">Central</a> |
|         | <a href="#">8.0.31</a> |                 | <a href="#">Central</a> |

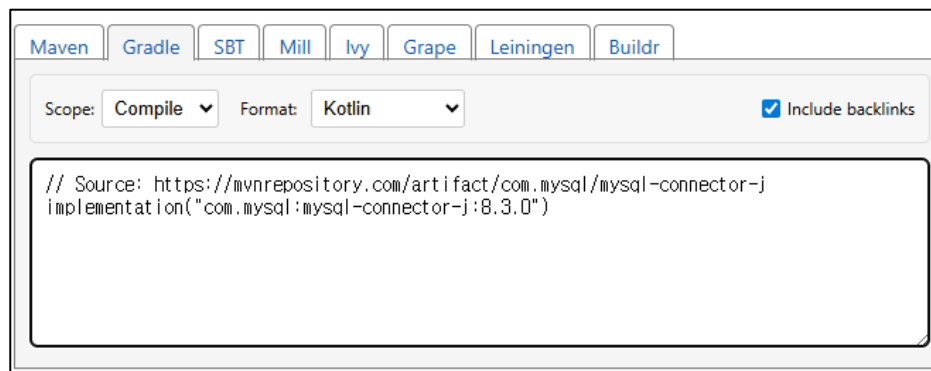


## spring

## ➤ MySQL 스프링 JDBC로 연결 하기



Build.gradle 더블클릭하여 의존성 주입한다.



```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'

    // Source: https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-jdbc
    implementation("org.springframework.boot:spring-boot-starter-jdbc")

    // Source: https://mvnrepository.com/artifact/com.mysql/mysql-connector-j
    implementation("com.mysql:mysql-connector-j")

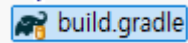
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    testImplementation 'org.springframework.boot:spring-boot-starter-thymeleaf-test'
    testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```





## spring

➤ MySQL 스프링 JDBC로 연결에 오류 출력 될 때 아래 코드로 변경하기



Build.gradle 더블클릭하여 의존성 주입한다.

```
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'  
    implementation 'org.springframework.boot:spring-boot-starter-web' // * 변경  
    //implementation 'org.springframework.boot:spring-boot-starter-webmvc'  
  
    // Source: https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-jdbc  
    implementation("org.springframework.boot:spring-boot-starter-jdbc")  
  
    // Source: https://mvnrepository.com/artifact/com.mysql/mysql-connector-j  
    implementation("com.mysql:mysql-connector-j")  
  
    developmentOnly 'org.springframework.boot:spring-boot-devtools'  
    testImplementation 'org.springframework.boot:spring-boot-starter-thymeleaf-test'  
    testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'  
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'  
}
```



## spring

## ➤ MySQL 스프링 JDBC로 연결 하기

- > Project and External Dependencies mysql jar 파일이 다운로드 되어 있는지 확인 한다.

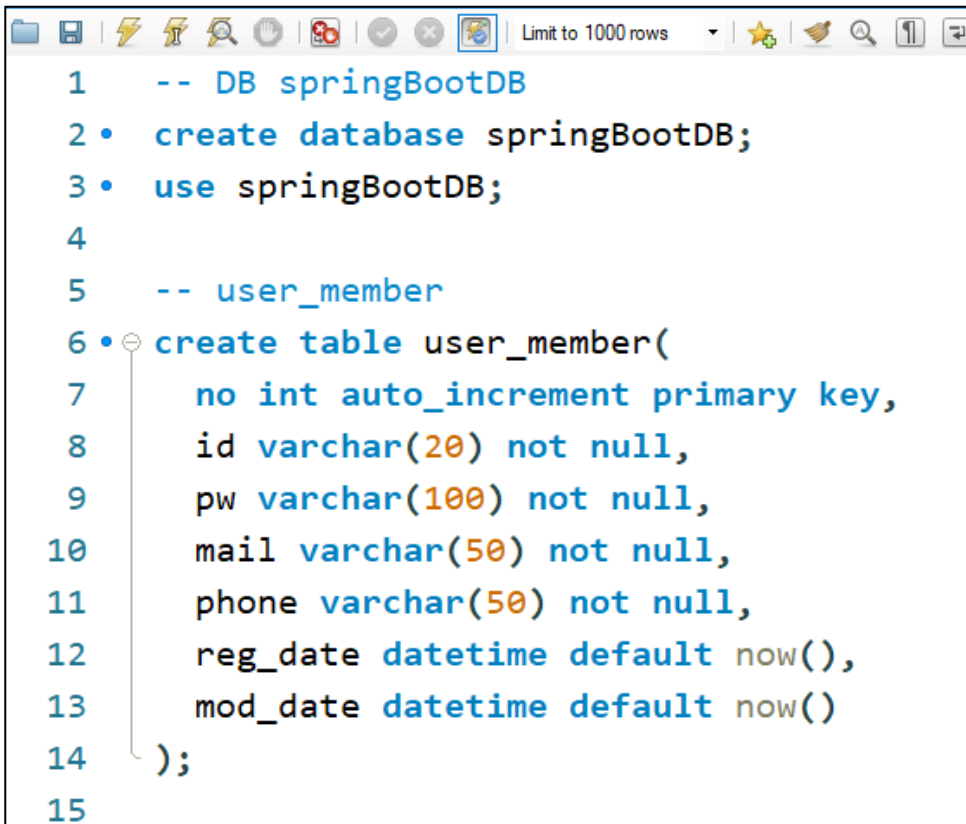
```
> log4j-api-2.25.3.jar - C:\Users\Admin\.gradle\cache\mo
> log4j-to-slf4j-2.25.3.jar - C:\Users\Admin\.gradle\cache\mo
> logback-classic-1.5.25.jar - C:\Users\Admin\.gradle\cache\mo
> logback-core-1.5.25.jar - C:\Users\Admin\.gradle\cache\mo
> micrometer-commons-1.17.0-M1.jar - C:\Users\Admin\.gradle\cache\mo
> micrometer-observation-1.17.0-M1.jar - C:\Users\Admin\.gradle\cache\mo
> mockito-core-5.21.0.jar - C:\Users\Admin\.gradle\cache\mo
> mockito-junit-jupiter-5.21.0.jar - C:\Users\Admin\.gradle\cache\mo
> mysql-connector-j-9.5.0.jar - C:\Users\Admin\.gradle\cache\mo
> objenesis-3.3.jar - C:\Users\Admin\.gradle\cache\mo
> opentest4j-1.3.0.jar - C:\Users\Admin\.gradle\cache\mo
> org.osgi.annotation.bundle-2.0.0.jar - C:\Users\Admin\.gradle\cache\mo
> org.osgi.annotation.versioning-1.1.2.jar - C:\Users\Admin\.gradle\cache\mo
> org.osgi.resource-1.0.0.jar - C:\Users\Admin\.gradle\cache\mo
> org.osgi.service.serviceloader-1.0.0.jar - C:\Users\Admin\.gradle\cache\mo
> slf4j-api-2.0.17.jar - C:\Users\Admin\.gradle\cache\mo
> snakeyaml-2.5.jar - C:\Users\Admin\.gradle\cache\mo
> spring-aop-7.0.3.jar - C:\Users\Admin\.gradle\cache\mo
> spring-beans-7.0.3.jar - C:\Users\Admin\.gradle\cache\mo
> spring-boot-4.1.0-M1.jar - C:\Users\Admin\.gradle\cache\mo
> spring-boot-autoconfigure-4.1.0-M1.jar - C:\Users\Admin\.gradle\cache\mo
> spring-boot-devtools-4.1.0-M1.jar - C:\Users\Admin\.gradle\cache\mo
> spring-boot-http-converter-4.1.0-M1.jar - C:\Users\Admin\.gradle\cache\mo
> spring-boot-jackson-4.1.0-M1.jar - C:\Users\Admin\.gradle\cache\mo
> spring-boot-jdbc-4.1.0-M1.jar - C:\Users\Admin\.gradle\cache\mo
```



## spring

## ➤ spring 데이터 베이스 연동 – 간단한 예제 MySQL Workbench 데이터 베이스 생성

springBootDB 데이터 베이스를 생성한다.



```
1  -- DB springBootDB
2 • create database springBootDB;
3 • use springBootDB;
4
5  -- user_member
6 • create table user_member(
7      no int auto_increment primary key,
8      id varchar(20) not null,
9      pw varchar(100) not null,
10     mail varchar(50) not null,
11     phone varchar(50) not null,
12     reg_date datetime default now(),
13     mod_date datetime default now()
14 );
15
```



## spring

## ➤ MySQL 스프링 JDBC로 연결 하기

- application.properties 파일에 MySQL 설정 변수를 설정 한다.

```
spring.application.name=com.green  
# Tomcat 서버를 내장되어 있다.  
# 원래 포트번호는 8080인데 => 8090으로 변경함  
# Tomcat  
server.port=8090
```

```
# Dev Tools  
spring.devtools.restart.enabled=true
```

```
# Thymeleaf #주석  
spring.thymeleaf.cache=false  
spring.thymeleaf.prefix=classpath:/templates/  
spring.thymeleaf.suffix=.html  
spring.thymeleaf.check-template-location=true
```

```
# MySQL  
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver  
spring.datasource.url=jdbc:mysql://localhost:3306/springBootDB  
spring.datasource.username=root  
spring.datasource.password=12345678
```



## ➤ Spring boot에서 데이터베이스 연동 - SQL과 JDBC

### ● SQL(Structured Query Language)

- 관계형 데이터베이스 관리 시스템에서 사용
- 데이터베이스 스키마 생성, 자료의 검색, 관리, 수정, 그리고 데이터베이스 객체 접근 관리를 위해 고안된 언어
- 데이터베이스로부터 정보를 추출하거나 갱신하기 위한 표준 대화식 프로그래밍 언어
  - 다수의 데이터베이스 관련 프로그램들이 SQL을 표준으로 채택

### ● JDBC (Java DataBase Connectivity)

- 관계형 데이터베이스에 저장된 데이터를 접근 및 조작할 수 있게 하는 API
- 다양한 DBMS에 대해 일관된 API로 데이터베이스 연결, 검색, 수정, 관리 등을 할 수 있게 함
- JDBC(Java Database Connectivity)란 자바환경에서 데이터베이스를 연결할 수 있는 기능을 의미합니다.
- 데이터베이스를 사용하려면 JVM 환경과 데이터베이스 환경 사이에 표준이 있어야 하는데 이러한 표준이 바로 JDBC입니다.



## ➤ Java 응용 프로그램과 JDBC 연결

DAO

```
try {  
    Connection conn =  
        DriverManager.getConnection("jdbc:mysql://localhost:3306/jspdatabase?serverTimezone=UTC", "root", "12345678");  
} catch (SQLException e) {  
    e.printStackTrace();  
}
```

- DriverManager는 자바 어플리케이션을 JDBC 드라이버에 연결시켜주는 클래스로서 getConnection() 메소드로 DB에 연결하여 Connection 객체 반환
- getConnection에서 jdbc: 이후에 지정되는 URL의 형식은 DB에 따라 다르므로 JDBC 문서를 참조
  - Mysql 서버가 같은 컴퓨터에서 동작하므로 서버 주소를 localhost로 지정
  - Mysql의 경우 디폴트로 3306 포트를 사용
  - jspdatabase는 앞서 생성한 DB의 이름
- "root"는 DB에 로그인할 계정 이름이며, "12345678"는 계정 패스워드



## ➤ 데이터 베이스 사용

### ▪ Statement 클래스

- SQL문을 실행하기 위해서는 Statement 클래스를 이용
- 주요 메서드

| 메소드                                | 설명                                                                       |
|------------------------------------|--------------------------------------------------------------------------|
| ResultSet executeQuery(String sql) | 주어진 sql문을 실행하고 결과는 ResultSet 객체에 반환                                      |
| int executeUpdate(String sql)      | INSERT, UPDATE, 또는 DELETE과 같은 sql문을 실행하고, sql 문 실행으로 영향을 받은 행의 개수나 0을 반환 |
| void close()                       | Statement 객체의 데이터베이스와 JDBC 리소스를 즉시 반환                                    |

- 데이터 검색을 위해 executeQuery() 메서드 사용
- 추가, 수정, 삭제와 같은 데이터 변경은 executeUpdate() 메서드 사용



## ➤ 데이터 베이스 사용

- ResultSet 클래스
  - SQL문 실행 결과를 얻어오기 위해서는 ResultSet 클래스를 이용
  - 현재 데이터의 행(레코드 위치)을 가리키는 커서(cursor)를 관리
  - 커서의 초기 값은 첫 번째 행 이전을 가리킴
  - 주요 메서드

| 메소드                                         | 설명                                                                                                                     |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <code>boolean first()</code>                | 커서를 첫 번째 행으로 이동                                                                                                        |
| <code>boolean last()</code>                 | 커서를 마지막 행으로 이동                                                                                                         |
| <code>boolean next()</code>                 | 커서를 다음 행으로 이동                                                                                                          |
| <code>boolean previous()</code>             | 커서를 이전 행으로 이동                                                                                                          |
| <code>boolean absolute(int row)</code>      | 커서를 지정된 행 row로 이동                                                                                                      |
| <code>boolean isFirst()</code>              | 첫 번째 행이면 true 반환                                                                                                       |
| <code>boolean isLast()</code>               | 마지막 행이면 true 반환                                                                                                        |
| <code>void close()</code>                   | ResultSet 객체의 데이터베이스와 JDBC 리소스를 즉시 반환                                                                                  |
| <code>Xxx getXxx(String columnLabel)</code> | Xxx는 해당 데이터 타입을 나타내며 현재 행에서 지정된 열 이름(columnLabel)에 해당하는 데이터를 반환한다. 예를 들어, int형 데이터를 읽는 메소드는 <code>getInt()</code> 이다.  |
| <code>Xxx getXxx(int columnIndex)</code>    | Xxx는 해당 데이터 타입을 나타내며 현재 행에서 지정된 열 인덱스(columnIndex)에 해당하는 데이터를 반환한다. 예를 들어, int형 데이터를 읽는 메소드는 <code>getInt()</code> 이다. |





## ➤ 데이터 베이스 사용

### ■ 테이블의 모든 데이터 검색

```
Statement stmt = conn.createStatement();  
ResultSet rs = stmt.executeQuery("select * from student");
```

- Statement의 executeQuery()는 SQL문의 실행하여 실행 결과를 넘겨줌
- 위의 SQL문의 student 테이블에서 모든 행의 모든 열을 읽어 결과를 rs에 저장

### ■ 특정 열만 검색

```
ResultSet rs = stmt.executeQuery("select name, id from student");
```

- 특정 열만 읽을 경우는 select문을 이용하여 특정 열의 이름 지정

### ■ 조건 검색

```
rs = stmt.executeQuery("select name, id, dept from student where id='0494013'");
```

- select문에서 where절을 이용하여 조건에 맞는 데이터 검색



## ➤ 데이터 베이스 사용

### ▪ 검색된 데이터 사용

```
while (rs.next()) {  
    System.out.println(rs.getString("name"));  
    System.out.println(rs.getString("id"));  
    System.out.println(rs.getString("dept"));  
}  
rs.close();
```

- Statement 객체의 `executeQuery()` 메소드
  - ResultSet 객체 반환
- ResultSet 인터페이스
  - DB에서 읽어온 데이터를 추출 및 조작할 수 있는 방법 제공
- `next()` 메소드
  - 다음 행으로 이동

|     |        |         |
|-----|--------|---------|
| 김철수 | 컴퓨터시스템 | 1091011 |
| 최고봉 | 멀티미디어  | 0792012 |
| 이기자 | 컴퓨터공학  | 0494013 |

next() : true

next() : true

next() : true

next() : true

next() : false



## ➤ 데이터 베이스 사용

### ▪ 데이터 추가

```
stmt.executeUpdate("insert into student (id, name, dept) values('0893012', '홍길동', '컴퓨터공학');");
```

- DB에 변경을 가하는 조작은 executeUpdate() 메소드 사용
- SQL문 수행으로 영향을 받은 행의 개수 반환

### ▪ 데이터 수정

```
stmt.executeUpdate("update student set id='0189011' where name= ' 홍길동'");
```

### ▪ 데이터 삭제

```
stmt.executeUpdate("delete from student where name= ' 홍길동'");
```



## ➤ 데이터 베이스 사용

### ▪ 데이터 추가

```
stmt.executeUpdate("insert into student (id, name, dept) values('0893012', '홍길동', '컴퓨터공학');");
```

- DB에 변경을 가하는 조작은 executeUpdate() 메소드 사용
- SQL문 수행으로 영향을 받은 행의 개수 반환

### ▪ 데이터 수정

```
stmt.executeUpdate("update student set id='0189011' where name= ' 홍길동'");
```

### ▪ 데이터 삭제

```
stmt.executeUpdate("delete from student where name= ' 홍길동'");
```



## spring

## ➤ spring에 데이터 베이스 연동 총 정리

### 1. JDBC란

Java 프로그램 내에서 데이터 베이스 내의 데이터를 사용하기 위해 SQL이 필요하게 되는데 이를 연결해 주는 응용프로그램 인터페이스

### 2. 연결순서

#### 1) Driver Loading

– DB와의 연결 위해 DBMC에서 제공하는 jar파일 Driver를 메모리에 적재

ex) `com.mysql.cj.jdbc.Driver`

#### 2) Connection

– 해당 드라이버를 사용하여 DB를 연결

ex) `String url = "jdbc:mysql://localhost:3306/springdb?serverTimezone=UTC";`

`Conn = DriverManager.getConnection(url,"root","12345678")`

#### 3) 쿼리 전달

– 쿼리를 DB로 전달하기 위해 `Statement`, `PreparedStatement` 객체를 생성

ex) `pstmt = conn.prepareStatement(sql);`

#### 4) 결과

– 전달된 쿼리의 수행으로 인한 반환 값

ex) `ResultSet rs = pstmt.executeQuery();`



spring

➤ **spring 데이터 베이스 연동 – 따라하는 간단한 예제 (회원가입)**

- MemberDao 클래스를 아래처럼 수정한다.

```
package com.green.member;

import java.sql.Connection;

@Repository
public class MemberDAO {

    @Autowired
    private DataSource dataSource;
```



spring

```
// 회원 추가
public int insertMember(MemberDTO mdto) {
    System.out.println("회원 추가 메소드 호출");

    String sql = "insert into user_member(id,pw,mail,phone) values (?, ?, ?, ?)";
    int result = 0;

    try (
        Connection conn = dataSource.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)
    ) {
        pstmt.setString(1, mdto.getId());
        pstmt.setString(2, mdto.getPw());
        pstmt.setString(3, mdto.getMail());
        pstmt.setString(4, mdto.getPhone());

        result = pstmt.executeUpdate();

    } catch (Exception e) {
        e.printStackTrace();
    }

    return result;
}
```



spring

## ➤ 화면 출력결과

회원가입

|        |             |
|--------|-------------|
| 아이디    | park        |
| 비밀번호   | ....        |
| 비밀번호확인 | ....        |
| Email  | p@naver.com |
| 전화번호   | 2345        |

회원가입

1 • `SELECT * FROM springbootdb.user_member;`

|   | no | id      | pw   | mail        | phone | reg_date            | mod_date            |
|---|----|---------|------|-------------|-------|---------------------|---------------------|
| 1 |    | kjb1045 | 1111 | k@naver.com | 1234  | 2026-01-26 05:39:00 | 2026-01-26 05:39:00 |
| 2 |    | park    | 2222 | p@naver.com | 2345  | 2026-01-26 05:46:28 | 2026-01-26 05:46:28 |
|   |    | NULL    | NULL | NULL        | NULL  | NULL                | NULL                |